API Security Risk Analysis Report

Project: API Security Assessment

Author: Mahamat ADAM ADAM

Table of Contents

Introduction 1
1. Scope of the Assessment 1
2. Tools Used 1
3. Tested API 1
4. Methodology 2
5. Security Findings 2
Finding 1: Open API Endpoint 2
Risk 2
Severity 2
Recommendation 2
Finding 2: Broken Object Level Authorization 3
Risk 3
Severity 4
Recommendation 4
Finding 3: Excessive Data Exposure 5
Risk 5
Severity 5
Recommendation 5
Finding 4: Missing Security Headers 6
Risk 6
Severity 6
Finding 5: Missing Rate Limiting 7
Conclusion 8

Introduction

Modern web and mobile applications rely heavily on APIs (Application Programming Interfaces) to exchange data between systems. Because APIs handle sensitive information and business logic, securing them is critical to prevent unauthorized access, data leaks, and abuse.

The objective of this project was to analyze a public test API and identify potential security risks by reviewing endpoints, responses, headers, and authentication mechanisms.

This analysis was performed in an ethical and controlled environment using a public API designed for learning and testing purposes.

1. Scope of the Assessment

The analysis focused on identifying common API security risks such as:

- ✓ Open or unauthenticated endpoints
- ✓ Excessive data exposure
- ✓ Weak authentication or authorization controls
- ✓ Missing security headers
- ✓ Lack of request rate limiting

The testing was performed in read-only mode without attempting exploitation or service disruption.

2. Tools Used

The following tools were used during the assessment:

Postman

Used to send API requests and analyze responses, headers, and status codes.

Web Browser

Used to view API responses and inspect network requests.

Documentation Tools

Microsoft Word were used to document the findings and produce the final report.

3. Tested API

The API selected for this analysis was a public testing API.

API Base URL:

https://jsonplaceholder.typicode.com

Example tested endpoints:

/users

/users/2

/users/7

This API provides sample user data which is useful for learning how APIs function and how security issues can appear in real-world scenarios.

4. Methodology

The following methodology was used during the assessment:

1. Review the API documentation and available endpoints
2. Send requests to endpoints using Postman
3. Analyze response data returned by the API
4. Inspect response headers
5. Evaluate authentication requirements
6. Identify potential security risks
7. Classify risk severity and propose remediation steps

5. Security Findings

Finding 1: Open API Endpoint

Description

During testing, the endpoint `/users` was accessed without any authentication requirements. The API returned user data immediately without requiring a login, API key, or token.

Example request:

GET /users

The server returned a successful response with status code:

200 OK

This means that anyone can access the endpoint and retrieve user data.

Risk

If this behavior existed in a real production system, attackers could freely collect user information without authorization.

Severity

Medium

Business Impact

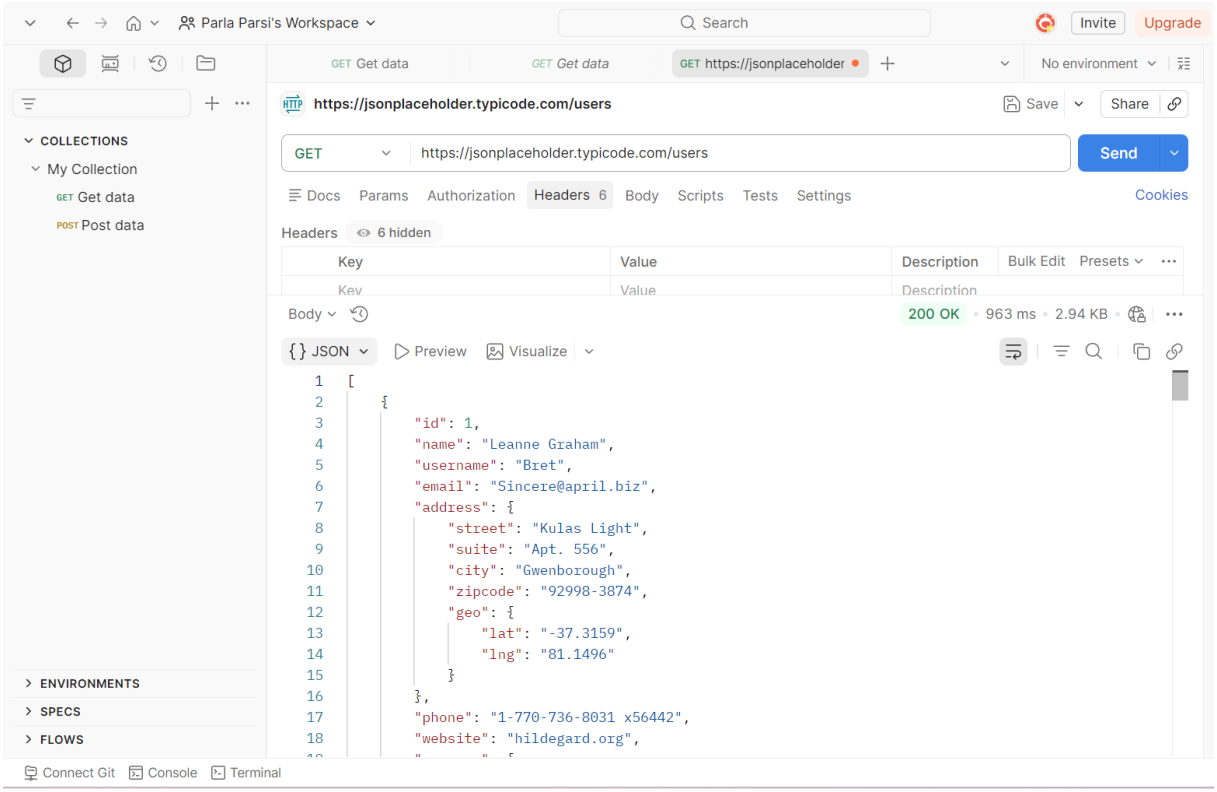
Unauthorized users could gather sensitive data which could be used for phishing, social engineering, or reconnaissance.

Recommendation

Implement proper authentication mechanisms such as:

- ✓ API keys

- ✓ OAuth tokens
- ✓ Access control policies



Capture showing:

Postman request to `/users`

API response

Status code `200 OK`

Finding 2: Broken Object Level Authorization

Description

The API allows access to specific user records by simply modifying the user ID in the URL.

Example requests tested:

/users/2

/users/7

Each request returned information about different users without verifying whether the requester was authorized to access that data.

Risk

This behavior could allow attackers to enumerate user IDs and access other users' data.

Severity

Medium

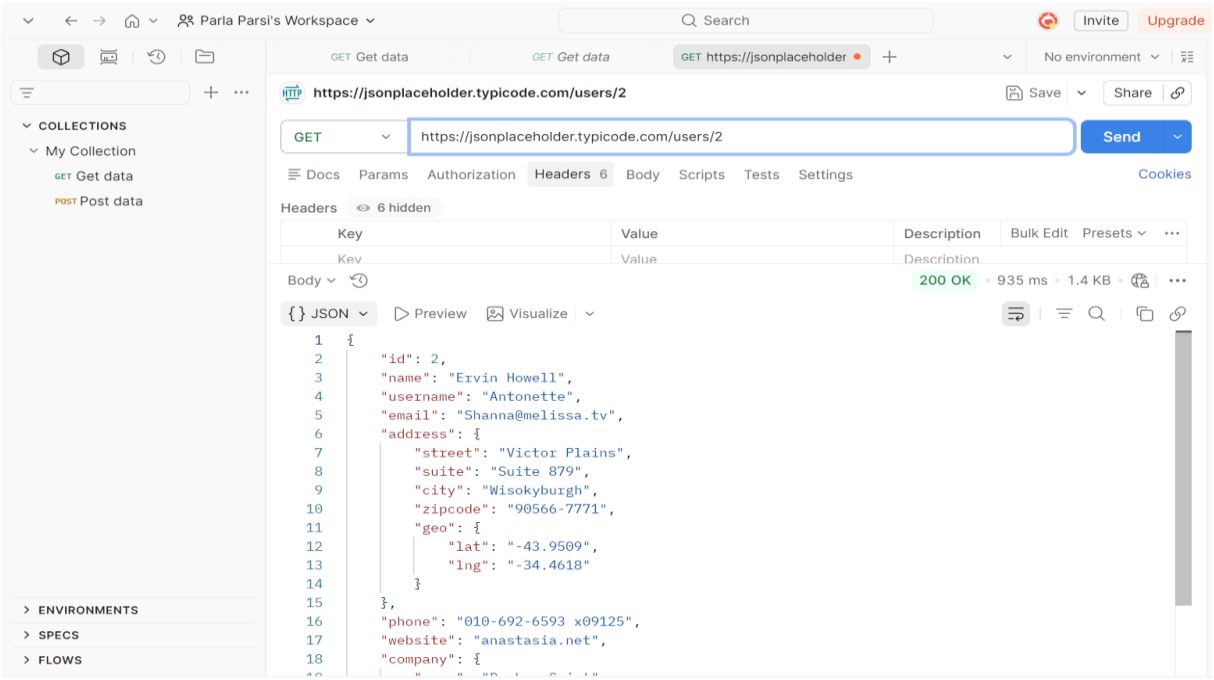
Business Impact

In real systems, this vulnerability may expose sensitive information belonging to other users.

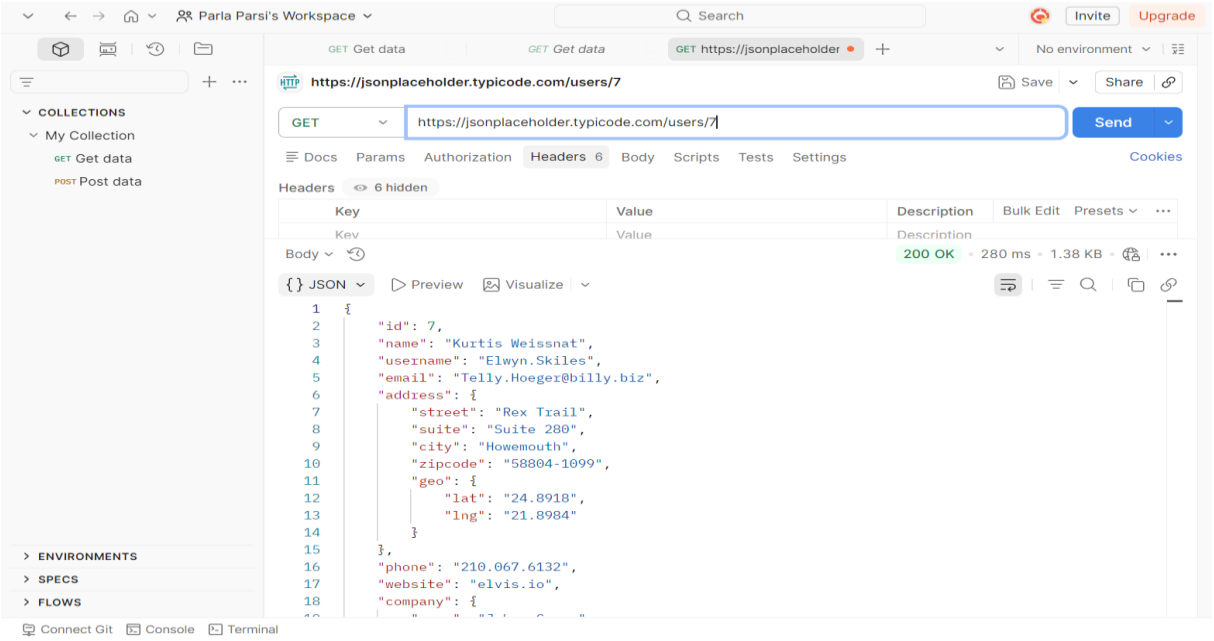
Recommendation

Implement proper authorization checks to ensure users can only access their own resources.

Capture showing:



Capture showing:



This demonstrates that the API allows enumeration.

Finding 3: Excessive Data Exposure

Description

The API response includes several fields that may expose unnecessary information.

Example fields returned:

- Email
- Phone
- Address
- Company

While this data is acceptable in a test environment, exposing too much information in production APIs can increase privacy risks.

Risk

Attackers may collect user data for phishing campaigns, identity profiling, or social engineering attacks.

Severity

Medium

Business Impact

Exposure of personal data can lead to privacy violations and reputational damage.

Recommendation

Only return necessary data fields and filter sensitive information at the server level.


```
{} JSON   Preview   Visualize
29        "email": "Shanna@melissa.tv",
30        "address": {
31            "street": "Victor Plains",
32            "suite": "Suite 879",
33            "city": "Wisokyburgh",
34            "zipcode": "90566-7771",
35            "geo": {
36                "lat": "-43.9509",
37                "lng": "-34.4618"
38            }
39        },
40        "phone": "010-692-6593 x09125",
41        "website": "anastasia.net",
42        "company": {
43            "name": "Deckow-Crist",
44            "catchPhrase": "Proactive didactic contingency",
45            "bs": "synergize scalable supply-chains"
46        }
```


Capture showing:

JSON response

Fields such as *email*, *phone*, *address*, *company*

Finding 4: Missing Security Headers

Description

Response headers were inspected to determine whether common HTTP security headers were implemented.

Typical security headers include:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- Strict-Transport-Security

These headers were not observed in the API response.

Risk

The absence of security headers may increase exposure to certain client-side attacks.

Severity

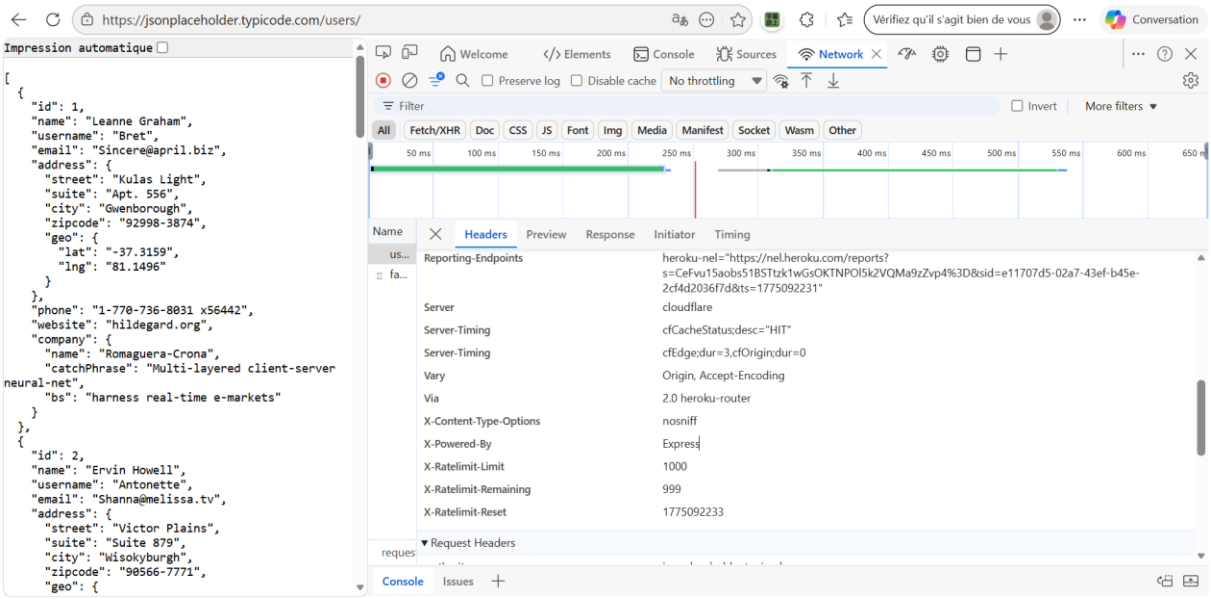
Low

Business Impact

Lack of security headers may reduce overall application security posture.

Recommendation

Implement standard HTTP security headers in API responses.



Capture showing:

Response headers returned by the API

Finding 5: Missing Rate Limiting

Description

Multiple requests were sent to the API endpoint in a short period of time. The API continued responding successfully without limiting the request frequency.

This suggests the absence of rate limiting mechanisms.

Risk

Without rate limiting, attackers could abuse the API by sending large volumes of requests.

Severity

Low

Business Impact

This could lead to:

- Service abuse
- Increased server load
- Potential denial-of-service conditions

Recommendation

Implement rate limiting to control excessive requests and protect server resources.

Conclusion

This assessment demonstrated how common API security risks can be identified through basic inspection and testing techniques.

Although the tested API is intentionally designed for educational purposes, the exercise highlights important security considerations that developers and organizations should address when designing APIs.

Key security improvements include implementing authentication, enforcing authorization checks, limiting data exposure, adding security headers, and applying rate limiting.

Understanding and addressing these issues is essential to protecting modern applications that rely heavily on APIs for communication and data exchange.